

assignment8

DSA Assignment #8: 🌲 (3/3)

Updated 2026-04-21 19:09 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M晴问9.7: 向下调整构建大顶堆

手搓堆, <https://sunnywhy.com/sfbj/9/7>

思路:

按照编号从大往小进行 `perc_down` 操作，注意到叶节点无需调整，因此可从序列 $n//2 - 1$ 的位置（对应序号最大的节点 $n-1$ 的父节点）开始调整，可减少部分常数。

代码:

```
class BinaryHeap:
    def __init__(self):
        self.heap = []

    def _perc_down(self, i):
        size = len(self.heap)
```

```

        while i < size // 2:
            min_idx = self.get_max_child(i)
            if self.heap[i] >= self.heap[min_idx]:
                break
            self.heap[i], self.heap[min_idx] = self.heap[min_idx],
self.heap[i]
            i = min_idx

def get_max_child(self, i):
    size = len(self.heap)
    if i * 2 + 2 >= size:
        return i * 2 + 1
    if self.heap[i * 2 + 1] < self.heap[i * 2 + 2]:
        return i * 2 + 2
    return i * 2 + 1

def heapify(self, not_heap):
    self.heap = not_heap[:]
    for i in range(len(self.heap) // 2 - 1, -1, -1):
        self._perc_down(i)

def main():
    n = int(input())
    a = list(map(int, input().split()))
    bh = BinaryHeap()
    bh.heapify(a)
    print(' '.join(map(str, bh.heap)))

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

21

22

def heapify(self, not_heap):

...

测试输入

提交结果

历史提交

完美通过

查看

100% 数据通过测试

详情

运行时长: 0 ms

M1722.执行交换操作后的最小汉明距离

dsu, <https://leetcode.cn/problems/minimize-hamming-distance-after-swap-operations/>

思路:

对可以交换的两个位置相当于连了一条边，最后构成的连通图中的位置可以任意交换，用并查集维护，同时用哈希表（defaultdict 嵌套）记录出现的数字个数，最后计算每个连通图里的 source与target的相同数字个数。

代码:

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        if x != self.parent[x]:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        x = self.find(x)
        y = self.find(y)
        if self.rank[x] < self.rank[y]:
            x, y = y, x
        self.parent[y] = x
        if self.rank[x] == self.rank[y]:
            self.rank[x] += 1
```

代码运行截图 (至少包含有"Accepted")



思路：

建树过程：使用heapq实现小根堆，每次选择两个weight值最小的节点合并，合并以后的权重等于子节点权重之和，再加入堆中。每个叶子节点的code等于从根节点开始的路径（左对应0，右对应1）。

代码：

```
import heapq

class Node:
    def __init__(self, weight : int, char = None):
        self.weight = weight
        self.char = char
        self.left = None
        self.right = None
    def __lt__(self, other):
        if self.weight == other.weight:
            return self.char < other.char
        return self.weight < other.weight

def build_huffman_tree(char_freq):
    heap = [Node(weight, char) for char, weight in char_freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node(left.weight + right.weight, min(left.char, right.char))
        merged.left = left
        merged.right = right
        heapq.heappush(heap, merged)
    return heap[0]

def encoding(root):
    char_code = {}
    stack = [(root, '')]
    while stack:
        node, path = stack.pop()
        if not node.left:
            char_code[node.char] = path
            continue
        stack.append((node.left, path + '0'))
        stack.append((node.right, path + '1'))
    return char_code

def decoding(code, root):
    node = root
    ans = []
```

```

for c in code:
    if not node.left:
        ans.append(node.char)
        node = root
    if c == '0':
        node = node.left
    else:
        node = node.right
ans.append(node.char)
return ''.join(ans)

def main():
    n = int(input())
    char_freq = {}
    for _ in range(n):
        data = input().split()
        char_freq[data[0]] = int(data[1])
    root = build_huffman_tree(char_freq)
    char_code = encoding(root)
    while True:
        try:
            s = input()
            if s[0] in ('0', '1'):
                print(decoding(s, root))
            else:
                print(''.join(char_code[c] for c in s))
        except EOFError:
            break

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#52355931提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import heapq

class Node:
    def __init__(self, weight : int, char = None):
        self.weight = weight
        self.char = char
        self.left = None
        self.right = None
    def __lt__(self, other):
        if self.weight == other.weight:
            return self.char < other.char
        return self.weight < other.weight

def build_huffman_tree(char_freq):
    heap = [Node(weight, char) for char, weight in char_freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node(left.weight + right.weight, min(left.char, right.char))
```

基本信息

#: 52355931
题目: 22161
提交人: 25n2500011619(xq)
内存: 3772kB
时间: 29ms
语言: Python3
提交时间: 2026-04-21 16:32:23

M晴问9.5: 平衡二叉树的建立

手搓AVL, <https://sunnywhy.com/sfbj/9/5/359>

思路:

实现AVL树的插入和先序遍历。

代码:

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.height = 1

class AVL:
    def __init__(self):
        self.root = None

    def insert(self, val):
        if self.root:
            self.root = self._insert(self.root, val)
        else:
            self.root = Node(val)
```

```

def _insert(self, node, val):
    if not node:
        return Node(val)
    if val < node.val:
        node.left = self._insert(node.left, val)
    else:
        node.right = self._insert(node.right, val)

    node.height = 1 + max(self._get_height(node.left),
self._get_height(node.right))

    balance = self._get_balance(node)
    if balance > 1:
        if self._get_balance(node.left) < 0:
            node.left = self._left_rotate(node.left)
            node = self._right_rotate(node)
        elif balance < -1:
            if self._get_balance(node.right) > 0:
                node.right = self._right_rotate(node.right)
            node = self._left_rotate(node)

    return node

def _left_rotate(self, node):
    nw = node.right
    node.right = nw.left
    nw.left = node
    node.height = 1 + max(self._get_height(node.left),
self._get_height(node.right))
    nw.height = 1 + max(self._get_height(nw.left),
self._get_height(nw.right))
    return nw

def _right_rotate(self, node):
    nw = node.left
    node.left = nw.right
    nw.right = node
    node.height = 1 + max(self._get_height(node.left),
self._get_height(node.right))
    nw.height = 1 + max(self._get_height(nw.left),
self._get_height(nw.right))
    return nw

def _get_height(self, node) -> int:
    if not node:
        return 0

```

```

        return node.height

def _get_balance(self, node) -> int:
    if not node:
        return 0
    return self._get_height(node.left) - self._get_height(node.right)

def preorder(self):
    if not self.root:
        return []
    res = []
    def dfs(node):
        if not node: return
        res.append(node.val)
        dfs(node.left)
        dfs(node.right)
    dfs(self.root)
    return res

def main():
    _ = int(input())
    seq = list(map(int, input().split()))
    avl = AVL()
    for x in seq:
        avl.insert(x)
    print(' '.join(map(str, avl.preorder()))))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

平衡二叉树的建立

通过数 704 提交数 1584 难度 中等 显示标签 ☆

题目描述

将 n 个互不相同的正整数先后插入到一棵空的AVL树中，求最后生成的AVL树的先序序列。

输入描述

第一行一个整数 n ($1 \leq n \leq 50$)，表示AVL树的结点个数；

第二行 n 个整数 a_i ($1 \leq a_i \leq 100$)，表示插入序列。

输出描述

输出 n 个整数，表示先序遍历序列，中间用空格隔开，行末不允许有多余的空格。

```
1 class Node:
2     def __init__(self, val):
3         self.val = val
4         self.left = None
5         self.right = None
6         self.height = 1
7
8 class AVL:
9     def __init__(self):
10        self.root = None
11
12    def insert(self, val):
13        if self.root:
14            self.root = self._inse
15        else:
16            self.root = Node(val)
17
18    def _insert(self, node, val):
```

测试输入

提交结果

历史提交

完美通过

100% 数据通过测试 详情

运行时长: 0 ms

M208.实现Trie（前缀树）

trie, <https://leetcode.cn/problems/implement-trie-prefix-tree/>

思路：

按照标准建 trie 树的思路，对于稀疏数据，可以用字典 Hash Map 来记录孩子节点；如果只有小写字母，也可以用邻接表来储存。

代码

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]
        node['#'] = '#'

    def search(self, word: str) -> bool:
```

```

node = self.root
for c in word:
    if c not in node:
        return False
    node = node[c]
return '#' in node

def startsWith(self, prefix: str) -> bool:
    node = self.root
    for c in prefix:
        if c not in node:
            return False
        node = node[c]
    return True

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

(至少包含有"Accepted")

← 全部提交记录

通过 16 / 16 个通过的测试用例

Jolly TuringO0u 提交于 2026.04.25 20:35

✦ 分析

✍ 写题解



面向在校学生的专享特惠

完成认证享 7 折 Plus 会员, 享受更多学业及职业成长帮助

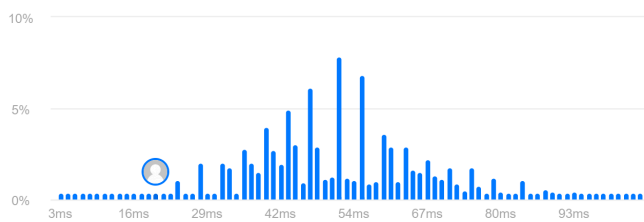


🕒 执行用时分布

20 ms | 击败 99.37% 🏆

💾 消耗内存分布

31.03 MB | 击败 92.46% 🏆



Python3 智能模式

```

1 class Trie:
2
3     def __init__(self):
4         self.root = {}
5
6     def insert(self, word: str) -> None:
7         node = self.root
8         for c in word:
9             if c not in node:
10                node[c] = {}
11                node = node[c]
12            node['#'] = '#'
13
14     def search(self, word: str) -> bool:

```

已存储

☑ 测试用例 | > 测试结果

通过 执行用时: 0 ms

☑ Case 1

输入

M307.区域和检索 - 数组可修改

segment tree, <https://leetcode.cn/problems/range-sum-query-mutable/>

思路:

使用非递归式的线段树实现方法, 用 $i+n$ 表示下标为 i 的点作为叶子节点, $i // 2$ 为 i 的父节点。建好树以后实现单点加、区间查询的功能。

代码

```
class NumArray:

    def __init__(self, nums: List[int]):
        self.n = len(nums)
        self.tree = [0] * (self.n << 1)
        for i in range(self.n):
            self.tree[i + self.n] = nums[i]
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[i << 1] + self.tree[i << 1 | 1]

    def update(self, index: int, val: int) -> None:
        index += self.n
        val -= self.tree[index]
        while index:
            self.tree[index] += val
            index >>= 1

    def sumRange(self, left: int, right: int) -> int:
        left += self.n
        right += self.n + 1
        res = 0
        while left < right:
            if left & 1:
                res += self.tree[left]
                left += 1
            if right & 1:
                right -= 1
                res += self.tree[right]
            left >>= 1
            right >>= 1
        return res
```

```
# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)
```

(至少包含有"Accepted")



2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

手搓堆和哈夫曼树相对思路比较简洁，Trie树用字典在稀疏数据时更快速，稠密数据可以用邻接表代替。这次学习老师的讲义时，发现堆和线段树都是采用非递归式的方式建立一棵完全二叉树，这与之前学的用节点与左右孩子的存储方法不同。用下标 i 表示父亲-子节点的关系，从而 $tree$ 数组天然就是层序遍历，这在空间复杂度上有明显的优化，而且实现起来也不复杂。

做最小汉明距离这道题时，学习了 `defaultdict` 的多重嵌套。

- `defaultdict(int)`：
`int` 是一个工厂函数，调用 `int()` 返回 `0`。因此 `defaultdict(int)` 相当于一个“值默认为 `0`”的字典，也就是一个计数器。
- `lambda: defaultdict(int)`：
这是一个匿名函数，每次调用时返回一个全新的 `defaultdict(int)` 对象。
- `sets = defaultdict(lambda: defaultdict(int))`：
外层 `defaultdict` 的默认值工厂是这个 `lambda`，因此当你访问 `sets[outer_key]` 而 `outer_key` 不存在时，会自动创建一个 `defaultdict(int)` 作为内层字典。

AVL树实现起来相对复杂，把左旋右旋操作、计算高度和平衡因子等函数都封装成函数会使代码变得简洁一些。